

机器如何做决策

How do Machines Make Decisions

孟庆鑫

人工智能学院



唐山学院
TANGSHAN UNIVERSITY

课件获取地址



GitHub



Gitee

如何做决策？

从数据中学习规则

决策树的构建流程

改进决策树的构建

借助群体智慧稳定决策

从错误中学习

世界充满决策

在现实世界中，我们每天都在做决策

例如：

- 银行是否批准贷款？
- 医生如何判断疾病？
- 电商平台是否推荐某个商品？

数据 → 判断 → 决策

案例：银行贷款审批

假设你在银行的风控部门工作

银行需要回答一个问题：

是否批准贷款？

银行可以观察客户的信息：

- 收入水平
- 是否有房
- 信用记录
- 工作稳定性

一个简单的决策

假设我们观察到：

- 收入高的人通常可以还款
- 收入低的人风险较大

于是一个简单规则可能是：

规则

如果收入 > 10000 ，则批准贷款

这是一个非常简单的决策规则

更复杂的决策

现实中情况往往更复杂：

- 收入高，但没有稳定工作
- 收入一般，但有房产
- 收入低，但信用记录很好

更复杂的决策

现实中情况往往更复杂：

- 收入高，但没有稳定工作
- 收入一般，但有房产
- 收入低，但信用记录很好

于是银行可能制定更复杂的规则：

贷款审批规则

- 如果 收入 > 10000
 - 如果 有房 $\rightarrow$ 批准
 - 否则 $\rightarrow$ 拒绝
- 如果 收入 ≤ 10000
 - 如果 信用良好 $\rightarrow$ 批准
 - 否则 $\rightarrow$ 拒绝

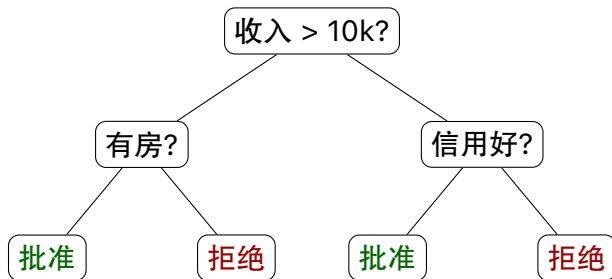
观察这个规则

我们仔细观察刚才的规则，会发现一个特点：

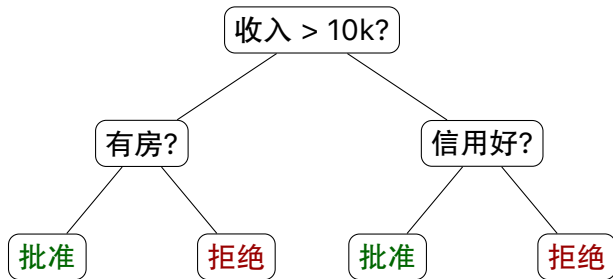
- 每一步都在问一个问题
- 根据答案进入不同路径
- 最终得到一个决策

决策 = 一系列问题

决策流程



决策流程



这就是决策树

- 内部节点：问题
- 分支：问题答案（左：Yes；右：No）
- 叶节点：最终决策

到目前为止，所有规则都是：

人类自己写出来的

那么一个自然的问题是：

机器能否从数据中学会这些规则？

如何做决策？

从数据中学习规则

决策树的构建流程

改进决策树的构建

借助群体智慧稳定决策

从错误中学习

如果没有规则

假设银行拥有大量历史数据：

ID	收入	房产	信用	是否违约
1	高	是	好	否
2	高	否	好	是
3	高	否	差	是
4	低	是	好	否
5	低	否	差	是
6	低	是	差	否
7	中	否	好	否
8	中	否	差	是

如果没有规则

假设银行拥有大量历史数据：

ID	收入	房产	信用	是否违约
1	高	是	好	否
2	高	否	好	是
3	高	否	差	是
4	低	是	好	否
5	低	否	差	是
6	低	是	差	否
7	中	否	好	否
8	中	否	差	是

问题变成能否从数据中自动发现决策规则

规则学习

机器学习的一个重要目标就是：

从数据中学习规则

如果学习成功，我们可以得到类似这样的规则：

学习到的规则

如果收入高且信用好，则批准贷款

一种自然的规则结构

在上一部分我们已经看到：

规则系统可以表示为：

树结构

因此机器学习的目标变成：

从数据中学习一棵决策树

学习一棵树

学习决策树可以理解为：

不断提出问题

例如：

- 收入是否高？
- 是否有房？
- 信用是否良好？

这些问题决定了树的结构

关键问题

在构建决策树时，最关键的问题是：

第一步应该问什么问题？

例如：

- 收入？
- 是否有房？
- 信用记录？

不同问题的效果

不同的问题可能产生完全不同的划分。

例如：

- 按收入划分
- 按信用划分
- 甚至按身份证尾号划分

某些问题可以很好地区分违约客户

某些问题则几乎没有帮助

什么是好问题？

于是我们得到一个核心问题：

什么是一个好的问题？

直觉上：

好问题应该能够更好地区分不同类别

好问题

在划分之前：

- 一半客户违约
- 一半客户不违约

这是一个非常不确定的情况

如果一个问题可以把客户分成：

- 一组几乎全部违约
- 一组几乎全部不违约

那么这个问题就非常有价值

例一：按信用划分

如果我们按信用划分数据：

信用高：90% 不违约，10% 违约

信用低：90% 违约，10% 不违约

两个子群体几乎是“纯”的

这显然是一个**非常好的问题**

例二：按身份证尾号划分

如果我们按一个无关特征划分：

奇数：50% 违约，50% 不违约

偶数：50% 违约，50% 不违约

划分之后系统仍然完全不确定

这显然是一个坏问题

好问题应减少系统的不确定性

从刚才的例子可以看出：

一个好的问题应该：

- 让不同类别尽量分开
- 让每个子群体尽量“纯”

换句话说：

好问题应该减少系统的不确定性

如何衡量不确定性？

我们需要一种方法来度量：

系统有多不确定

在信息论中，这种不确定性有一个经典度量：

熵（Entropy）

熵 (Entropy)

在前面的课程中我们已经看到：

$$H(X) = - \sum_x P(x) \log P(x)$$

- 熵越大：系统越不确定，分布越均匀
- 熵越小：系统越确定，某个类别的概率越接近 1

决策树的思想

现在我们可以形式化“好问题”的度量：

- 如果当前数据集的不确定性是 $H(Y)$
- 按问题 A 划分之后，剩余的不确定性是 $H(Y | A)$

决策树的思想

现在我们可以形式化“好问题”的度量：

- 如果当前数据集的不确定性是 $H(Y)$
- 按问题 A 划分之后，剩余的不确定性是 $H(Y | A)$

于是我们定义：

信息增益

$$\text{Gain}(A) = H(Y) - H(Y | A)$$

含义非常简单：

问题 A 减少了多少不确定性

决策树的核心原则

选择信息增益最大的问题

以最大程度减少不确定性

重复上述过程至子群体纯

补充：条件熵的计算

- 联合概率等于条件概率乘以边缘概率： $P(a, y) = P(y | a) P(a)$
- 取负对数后得到 $-\log P(a, y) = -\log P(y | a) - \log P(a)$
- 再对两侧同时求期望，得到

$$\begin{aligned}& - \sum_{a,y} P(a, y) \log P(a, y) \\&= - \sum_{a,y} P(a, y) \log P(y | a) - \sum_{a,y} P(a, y) \log P(a) \\&= - \sum_a P(a) \sum_y P(y | a) \log P(y | a) - \sum_a P(a) \log P(a) \sum_y P(y | a)\end{aligned}$$

- 红色为 $H(A, Y)$ 绿色为 $H(Y | A)$ 蓝色为 $H(A)$

如何做决策？

从数据中学习规则

决策树的构建流程

改进决策树的构建

借助群体智慧稳定决策

从错误中学习

训练数据

ID	收入	房产	信用	是否违约
1	高	是	好	否
2	高	否	好	是
3	高	否	差	是
4	低	是	好	否
5	低	否	差	是
6	低	是	差	否
7	中	否	好	否
8	中	否	差	是

目标：

从数据中“长出”一棵决策树

整体不确定性

整体违约情况：

- 是：4
- 否：4

$$H(Y) = -\frac{4}{8} \log_2 \frac{4}{8} - \frac{4}{8} \log_2 \frac{4}{8} = 1$$

系统完全不确定

候选问题

第一步，我们需要选择一个“好问题”：

- 收入？
- 房产？
- 信用？

比较信息增益

计算：按“收入”划分

- 收入 = 高 (3 个, 违约情况: 否、是、是)

$$H_{\text{收入高}} = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} \approx 0.92$$

- 收入 = 中 (2 个: 否、是)

$$H_{\text{收入中}} = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$$

- 收入 = 低 (3 个: 否、是、否) $\Rightarrow H_{\text{收入低}} \approx 0.92$

加权:
$$H(Y | \text{收入}) = \frac{3}{8} H_{\text{收入高}} + \frac{2}{8} H_{\text{收入中}} + \frac{3}{8} H_{\text{收入低}} \approx 0.94$$

$$\text{Gain}(\text{收入}) = H(Y) - H(Y | \text{收入}) \approx 0.06$$

计算：按“房产”划分

- 房产 = 有 (3 个, 违约情况: 否、否、否) $\implies H_{\text{有房产}} = 0$
- 房产 = 无 (5 个: 是、是、是、否、是)

$$H_{\text{无房产}} = -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \approx 0.72$$

加权: $H(Y | \text{房产}) = \frac{3}{8} H_{\text{有房产}} + \frac{5}{8} H_{\text{无房产}} \approx 0.45$

$$\text{Gain}(\text{房产}) = H(Y) - H(Y | \text{房产}) \approx 0.55$$

计算：按“信用”划分

- 信用 = 好 (4 个, 违约情况: 否、是、否、否)

$$H_{\text{信用好}} = -\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4} \approx 0.81$$

- 信用 = 差 (4 个: 是、是、否、是) $\implies H_{\text{信用差}} \approx 0.81$

加权: $H(Y | \text{信用}) = \frac{4}{8} H_{\text{信用好}} + \frac{4}{8} H_{\text{信用差}} \approx 0.81$

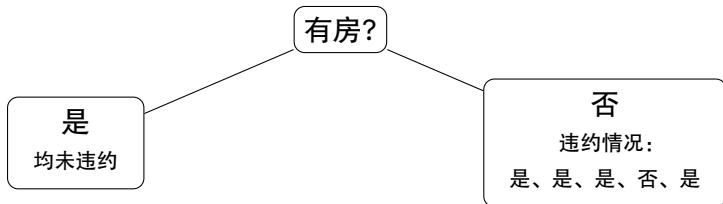
$$\text{Gain}(\text{信用}) = H(Y) - H(Y | \text{信用}) \approx 0.19$$

选择最优问题

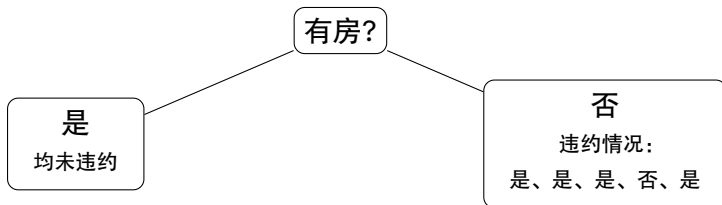
特征	收入	房产	信用
信息增益	0.06	0.55	0.19

选择：房产

第一层决策树

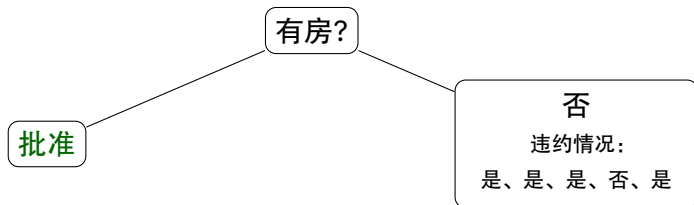


第一层决策树



左边已经纯，右边需要继续划分

第一层决策树



左边为批准，右边需要继续划分

子问题 (房产 = 否)

ID	收入	房产	信用	是否违约
2	高	否	好	是
3	高	否	差	是
5	低	否	差	是
7	中	否	好	否
8	中	否	差	是

子问题的不确定性:

$$H(Y) = -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \approx 0.72$$

备选问题 (特征): 收入、信用

子问题划分（选择“收入”）

- 收入 = 高（2 个，违约情况：是、是） $\implies H_{\text{收入高}} = 0$
- 收入 = 中（2 个：否、是） $\implies H_{\text{收入中}} = 1$
- 收入 = 低（1 个：是） $\implies H_{\text{收入低}} = 0$

$$H(Y | \text{收入}) = \frac{2}{5}H_{\text{收入高}} + \frac{2}{5}H_{\text{收入中}} + \frac{1}{5}H_{\text{收入低}} = 0.4$$

$$\text{Gain}(\text{收入}) = H(Y) - H(Y | \text{收入}) \approx 0.32$$

子问题划分（选择“信用”）

- 信用 = 好（2个：是、否） $\implies H_{\text{信用好}} = 1$
- 信用 = 差（3个：是、是、是） $\implies H_{\text{信用差}} = 0$

$$H(Y | \text{信用}) = \frac{2}{5}H_{\text{信用好}} + \frac{3}{5}H_{\text{信用差}} = 0.4$$

$$\text{Gain}(\text{信用}) = H(Y) - H(Y | \text{信用}) \approx 0.32$$

当信息增益相同

我们发现：

$$\text{Gain}(\text{收入}) = \text{Gain}(\text{信用}) \approx 0.32$$

两个问题一样好

当信息增益相同

我们发现：

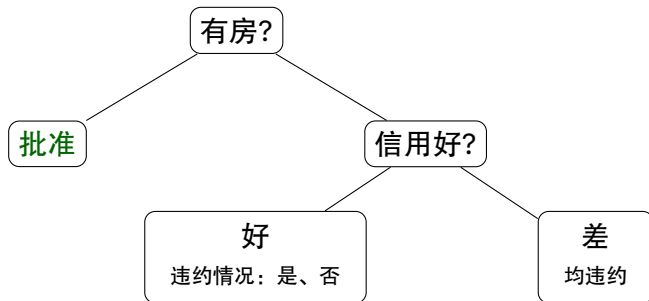
$$\text{Gain}(\text{收入}) = \text{Gain}(\text{信用}) \approx 0.32$$

两个问题一样好

为了让决策树继续生长

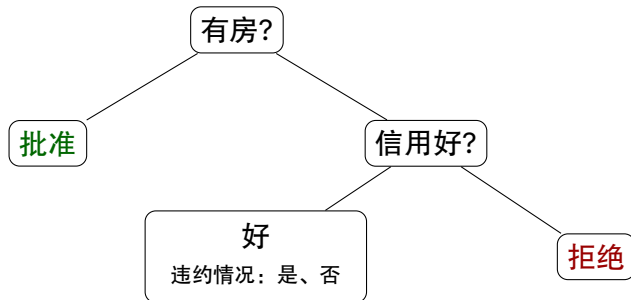
我们可以任选一个特征

第二层决策树（选择“信用”）



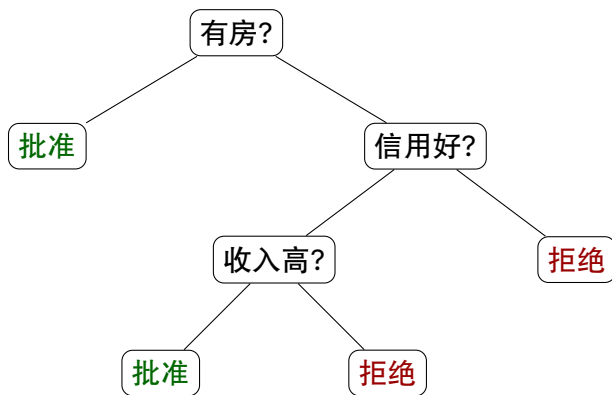
右边已经纯，左边需要继续划分

第二层决策树（选择“信用”）



左边按“收入”划分

最终三层决策树



总结决策树的生长过程

选一个好问题 $\longrightarrow$ 划分数据 $\longrightarrow$ 在子数据上重复，直到节点足够纯

总结起来：

- 使用信息增益选择特征
- 递归构建树结构

总结决策树的生长过程

选一个好问题 $\longrightarrow$ 划分数据 $\longrightarrow$ 在子数据上重复，直到节点足够纯

总结起来：

- 使用信息增益选择特征
- 递归构建树结构

这正是经典的：

ID3 (Iterative Dichotomiser 3) 算法

ID3 的命名

命名成份	含义
Iterative	以自顶向下的方式递归拆分数据
Dichotomiser	将属性拆分为更小的组以构建树
3	Ross Quinlan 创建的这种算法的第三个版本

如何做决策？

从数据中学习规则

决策树的构建流程

改进决策树的构建

借助群体智慧稳定决策

从错误中学习

信息增益真的完美吗？

现在，我们引入一个新特征：

客户 ID

它的特点是：

每个样本的 ID 都不同

那么，用它来划分会发生什么？

计算客户 ID 的信息增益

使用 ID 划分：

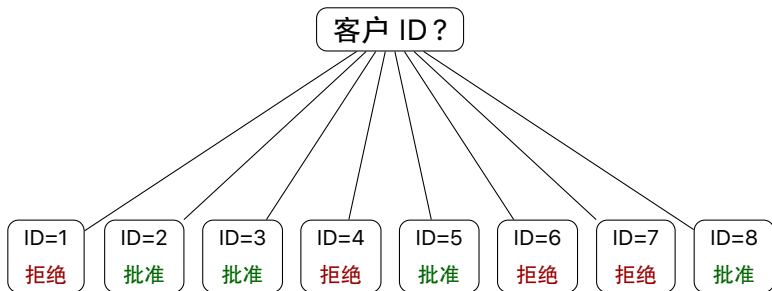
- 每个子集只有一个样本
- 每个子集完全纯

这意味着

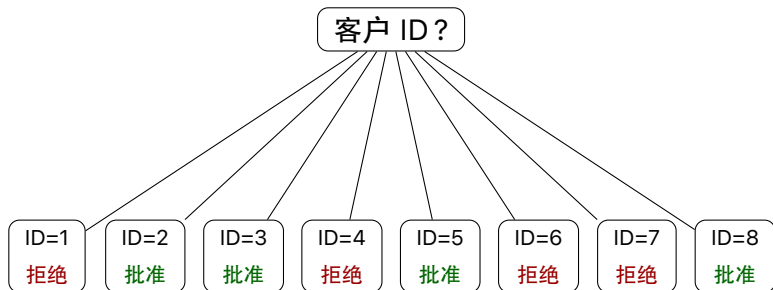
$$H(Y \mid \text{ID}) = 0 \quad \text{Gain}(\text{ID}) = 1$$

信息增益最大！

引入客户 ID 后的决策树



引入客户 ID 后的决策树



问题：新客户 ID9 来了，你要批准还是拒绝？

问题出在哪里？

ID3 的策略非常清晰：

选择信息增益最大的特征

谁最能降低不确定性，就选谁

然而

信息增益倾向选择取值数量多

划分更细的特征

问题的本质

它没有学会规律

它只是“记住了”数据

好划分 $\neq$ 好决策

如何修正？

由于信息增益偏好“过度细分”的特征，因此需要一个修正：

对划分本身进行惩罚

- 我们已经有一个熵 $H(Y)$ ，它描述的是标签分布的均匀性
- 我们引入一个度量，它要衡量划分有多“碎”

划分的熵

如果一个特征 A 把数据划分为多个子集：

$$D_1, D_2, \dots, D_k$$

每个子集的概率是：

$$p_i = \frac{|D_i|}{|D|}$$

那么我们可以定义：

$$H(A) = - \sum_i p_i \log p_i$$

这就是 SplitInfo，它衡量划分有多“碎”

如何平衡我们的需求？

我们需要：

- 信息增益大
- 划分不过于“细碎”

一种自然的方式是：

$$\frac{\text{信息增益}}{\text{划分的“细碎”程度}}$$

如何平衡我们的需求？

我们需要：

- 信息增益大
- 划分不过于“细碎”

一种自然的方式是：

$$\frac{\text{信息增益}}{\text{划分的“细碎”程度}}$$

于是得到：

信息增益率

$$\text{GainRatio}(A) = \frac{\text{Gain}(A)}{H(A)}$$

更深的理解

信息增益

只关注“结果”，只专注于“拟合数据”

信息增益率

不仅关注“结果”，也开始约束“过程”

不仅要拟合数据，也要控制划分的“细碎”程度

现在，我们可以用
信息增益率 GainRatio
取代
信息增益 Gain
来生成一棵树

这样的树真的可靠吗？

极端情况

如果我们不断划分：

- 每个叶节点只剩一个样本
- 训练数据将被完全正确分类

极端情况

如果我们不断划分：

- 每个叶节点只剩一个样本
- 训练数据将被完全正确分类

它是在学习规律，还是在记忆数据？

回到贷款案例

假设在某个节点，我们继续划分：

问题：客户 ID？

结果：

- 每个样本进入不同分支
- 每个叶节点完全“纯”

回到贷款案例

假设在某个节点，我们继续划分：

问题：客户 ID？

结果：

- 每个样本进入不同分支
- 每个叶节点完全“纯”

模型不再学习规律，而是在记忆数据

这就是过拟合

过拟合的本质

树越复杂
越容易记住“偶然”
反而越不可靠

一个新的问题

既然如此：树应该长到哪里停止？哪些分支是“有用的”？

一个新的问题

既然如此：树应该长到哪里停止？哪些分支是“有用的”？

直觉上，

一个分支如果不能
带来真正的决策提升
那么它就不应该存在

剪枝的思想

我们先用信息增益率 GainRatio 来生成一棵树
再对这棵树进行“剪枝”

如何判断一个分支是否有用？

要看这个分支是否真的提升了决策能力

一个重要原则

不要用训练数据评判自己

因为训练数据已经被“记住”

一个重要原则

不要用训练数据评判自己

因为训练数据已经被“记住”

具体来说，我们把数据分成两部分：

- 一部分用来训练（建树）
- 一部分用来测试（验证）

用“没见过的数据”评估决策树

剪枝操作

对于某一个节点

我们有两种选择：

- 保留子树（复杂模型）
- 删除子树（变成叶节点）

然后比较哪一个在验证数据上更好？

- 如果剪枝后效果更好（或差不多），就剪掉这个分支
- 否则保留

剪枝的深层理解

在“拟合数据”和“控制复杂度”之间平衡

C4.5

- 用信息增益率控制划分的“细碎”程度
- 用剪枝控制复杂度使之不要过拟合

两者共同作用，产生的决策树算法即为

C4.5 算法

C4.5

- 用信息增益率控制划分的“细碎”程度
- 用剪枝控制复杂度使之不要过拟合

两者共同作用，产生的决策树算法即为

C4.5 算法

命名成份	含义
C	普遍认为 C 表示该算法最初是用 C 语言实现的
4.5	软件版本号

决策树的生长是在追求确定性
而剪枝
是在对抗这种过度的确定性

如何做决策？

从数据中学习规则

决策树的构建流程

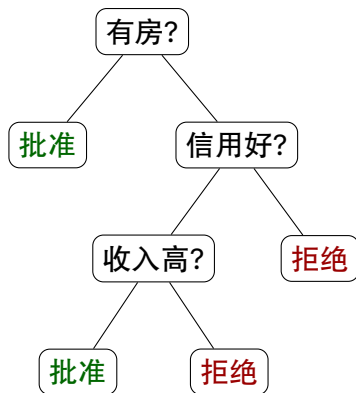
改进决策树的构建

借助群体智慧稳定决策

从错误中学习

C4.5 决策树

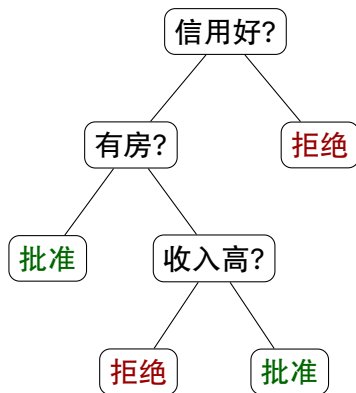
ID	收入	房产	信用	是否违约
1	高	是	好	否
2	高	是	好	是
3	高	否	差	是
4	低	是	好	否
5	低	是	差	是
6	低	是	差	否
7	中	否	好	否
8	中	否	差	是



有兴趣的同学可尝试验证这一结果

C4.5 决策树（数据表微小改动）

ID	收入	房产	信用	是否违约
1	高	是	好	否
2	高	是	好	是
3	高	否	差	是
4	低	是	好	否
5	低	否	差	是
6	低	是	差	★ 是 ★
7	中	否	好	否
8	中	否	差	是



有兴趣的同学可尝试验证这一结果

不稳定性

我们只是对数据做了很小的改变

然而

整棵树的拓扑结构却发生了全局性的重建

这意味着

某个样本的微小改变

足以推翻现有的决策逻辑

决策树模型对数据如此敏感

我们是否应该相信它？

换一种思路：用群体智慧

如果单一决策容易受到偶然因素影响

那么我们不要依赖单一决策

我们希望用群体智慧

来对抗单一决策的不稳定性

目标

我们要训练产生多棵决策树
并通过投票表决来确定最终决策

问题：如何得到不同的树？

如果我们用同一份数据训练
那么每棵决策树都一样

如何让每棵树“看起来不一样”？
我们需要制造不同数据

制造不同数据

原始数据: ID1, ID2, ID3, ..., ID8

制造不同数据

原始数据: ID1, ID2, ID3, ..., ID8

对原始数据随机抽样

得到一份新数据: ID2, ID5, ID5, ID7, ID1, ID3, ID8, ID2

制造不同数据

原始数据: ID1, ID2, ID3, ..., ID8

对原始数据随机抽样

得到一份新数据: ID2, ID5, ID5, ID7, ID1, ID3, ID8, ID2

再来一次: ID4, ID4, ID6, ID1, ID3, ID7, ID7, ID8

制造不同数据

原始数据: ID1, ID2, ID3, ..., ID8

对原始数据随机抽样

得到一份新数据: ID2, ID5, ID5, ID7, ID1, ID3, ID8, ID2

再来一次: ID4, ID4, ID6, ID1, ID3, ID7, ID7, ID8

再来一次: ID1, ID5, ID3, ID8, ID2, ID2, ID7, ID4

...

制造不同数据

原始数据: ID1, ID2, ID3, ..., ID8

对原始数据随机抽样

得到一份新数据: ID2, ID5, ID5, ID7, ID1, ID3, ID8, ID2

再来一次: ID4, ID4, ID6, ID1, ID3, ID7, ID7, ID8

再来一次: ID1, ID5, ID3, ID8, ID2, ID2, ID7, ID4

...

- 这些数据有的样本被重复抽中, 有的样本则完全消失
- 每棵树看到的世界不同

这种方法称为:

Bootstrap 抽样

问题仍然存在

即使数据不同
决策树仍然可能选择同一个“最佳特征”

为了让每棵树“看起来不一样”
我们还需要限制每棵树只看一部分特征

抽取不同特征

原本：收入 / 房产 / 信用

现在：随机选 2 个

抽取不同特征

原本：收入 / 房产 / 信用

现在：随机选 2 个

例如

- 第一组数据随机分配特征 收入 / 房产
- 第二组数据随机分配特征 房产 / 信用
- 第三组数据随机分配特征 收入 / 信用

...

这一步叫做：

随机特征选择

结果会怎样？

不同的树：

- 看到不同的数据（ Bootstrap ）
- 使用不同的特征（ 随机特征 ）

树开始真正不同

结果会怎样？

不同的树：

- 看到不同的数据（ Bootstrap ）
- 使用不同的特征（ 随机特征 ）

树开始真正不同

这些树构成的森林被称为**随机森林**

随机森林的本质

通过制造差异换取稳定

如何做决策？

从数据中学习规则

决策树的构建流程

改进决策树的构建

借助群体智慧稳定决策

从错误中学习

随机森林通过“并行的智慧”消除不稳定

是否有另一种可能：

如果我们不追求“人多力量大”

而是追求“步步为营、查漏补缺”？

现实中的学习过程

当你在准备考试时：

- ① 第一轮复习，你会发现哪些题做错了，哪些知识点掌握不牢
- ② 第二轮复习，你会把更多时间放在这些“容易出错”的地方
- ③ 如此反复，每一轮都在弥补上一轮的不足

现实中的学习过程

当你在准备考试时：

- ① 第一轮复习，你会发现哪些题做错了，哪些知识点掌握不牢
- ② 第二轮复习，你会把更多时间放在这些“容易出错”的地方
- ③ 如此反复，每一轮都在弥补上一轮的不足

学习不是一次完成的，而是不断修正的过程

把这种过程交给机器

如果让机器来学习，能不能也这样做？

先做一次整体判断

再专门去修正“做得不好的地方”

一轮一轮地，把错误逐步消除

把这种过程交给机器

如果让机器来学习，能不能也这样做？

先做一次整体判断

再专门去修正“做得不好的地方”

一轮一轮地，把错误逐步消除

这是一种完全不同于“并行平均”的思路

我们训练的，不再是一棵“完美的树”
而是一系列逐步改进的模型

- 第一轮：给出一个初步判断
- 第二轮：专门修正第一轮做得不好的地方
- 第三轮：继续修正已有模型的不足
- 每一轮，都是在“补漏洞”

如何让模型“关注错误”？

机器怎么知道
哪些地方是“还没学好”的？

如何让模型“关注错误”？

机器怎么知道
哪些地方是“还没学好”的？

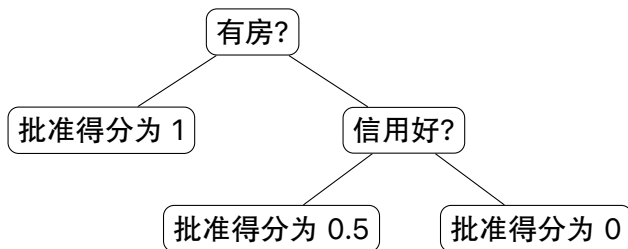
加权：改变数据的“重要性”

- 做得好的样本 → 权重下降，影响减小
- 做得差的样本 → 权重上升，影响增大

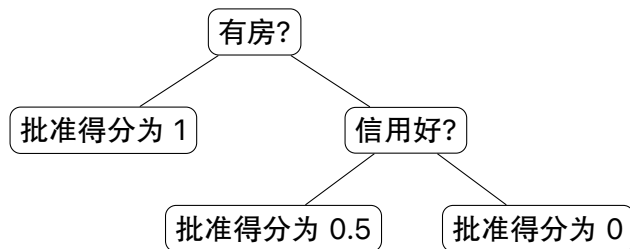
数据的重新理解

ID	X^1	X^2	X^3	Y	权重 w_t
1	2	1	1	0	w_t^1
2	2	0	1	1	w_t^2
3	2	0	0	1	w_t^3
4	0	1	1	0	w_t^4
5	0	0	0	1	w_t^5
6	0	1	0	0	w_t^6
7	1	0	1	0	w_t^7
8	1	0	0	1	w_t^8

第一步：建立初始模型



第一步：建立初始模型



运行 C4.5 得到的第一棵决策树可以用一个得分函数来表示：

$$h_1(x) \in [0, 1]$$

例如， $x = (0, 0, 1)$ 表示 “低收入 / 无房 / 信用好”

根据决策规则，批准贷款得分为 $h_1(x) = 0.5$

重新理解决策模型

假设我们已经有一系列的决策树： $h_1(x), \dots, h_T(x)$

我们不再直接输出“决策”，而是输出一个评分函数

$$F_T(x) = \sum_{t=1}^T \alpha_t h_t(x)$$

其中 α_t 表示这棵树的“可信度”：

- 树越准确 $\longrightarrow \alpha_t$ 越大
- 树越差 $\longrightarrow \alpha_t$ 越小

度量第一棵树的“偏差”

例如 ID=2 的样本 $x_2 = (2, 0, 1)$ ，决策树给出 $h_1(x_2) = 0.5$

但真实标签是 $y_2 = 1$

因此自然的偏差是： $|y_i - h_1(x_i)|$ ，或者更一般的函数 $\ell(y_i, h_1(x_i))$

- 偏差小：说明预测接近真实概率
- 偏差大：说明这一点“还没学好”
- 偏差 = 下一轮学习的重点

到目前为止，我们已知什么？

第一棵树 $h_1(x)$ 已经给出了一个初步概率估计

对于每个样本 i ，我们也知道 偏差 = $\ell(y_i, h_1(x_i))$

到目前为止，我们已知什么？

第一棵树 $h_1(x)$ 已经给出了一个初步概率估计

对于每个样本 i ，我们也知道 偏差 = $\ell(y_i, h_1(x_i))$

接下来该做什么？

接下来的路线图

我们接下来要完成三件事情：

- ① 根据偏差，重新分配样本权重
- ② 在新的权重下，训练第二棵树 $h_2(x)$
- ③ 把 $h_2(x)$ 组合进已有模型，修正总分

接下来的路线图

我们接下来要完成三件事情：

- ① 根据偏差，重新分配样本权重
- ② 在新的权重下，训练第二棵树 $h_2(x)$
- ③ 把 $h_2(x)$ 组合进已有模型，修正总分

Boosting 的本质就是反复完成这三步

第一件事：更新样本权重

- 偏差大的样本 $\rightarrow$ 应该更重要

$$\ell(y_i, h_t(x_i)) \uparrow \longrightarrow w_{t+1}^i \uparrow$$

- 偏差小的样本 $\rightarrow$ 可以降低关注

$$\ell(y_i, h_t(x_i)) \downarrow \longrightarrow w_{t+1}^i \downarrow$$

但具体“怎么变”？这是一个数学问题

第二件事：权重改变后，树也会改变

现在数据不再是“等权”的，每个样本都有权重 w_t^i ，这意味着：

- 重要样本会“被重复看到”
- 不重要样本影响变小

第二件事：权重改变后，树也会改变

现在数据不再是“等权”的，每个样本都有权重 w_t^i ，这意味着：

- 重要样本会“被重复看到”
- 不重要样本影响变小

决策树的训练规则必须随之改变

- 在普通决策树中：信息增益依赖于数样本数量来计算
- 现在变为：信息增益通过累加样本权重来

从统计某一类有多少个样本
变成统计这一类的总权重

第三件事：如何计入总分？

在第 t 轮，我们已有当前模型 $F_{t-1}(x)$

现在加入新一棵树 $h_t(x)$ ，总分为

$$F_t(x) = F_{t-1}(x) + \alpha_t h_t(x)$$

α_t 是唯一未知量，它表示这棵树的“可信度”：

- 树越准确 $\rightarrow \alpha_t$ 越大
- 树越差 $\rightarrow \alpha_t$ 越小

我们可以通过一个变换得到概率：
$$P(Y = 1 \mid X = x) = \frac{1}{1 + e^{-F_t(x)}}$$

现在问题被彻底拆解清楚了

我们接下来要解决下面两个数学问题

- ① 如何根据偏差更新权重 w_{t+1} ?
- ② 如何确定每棵树的权重 α_t ?

关键洞察

看似两个问题，其实是同一件事的两个侧面

- 问题 1：数据层如何更新权重 w_{t+1} ?

本质是让权重关注“误差”的同时，对当前权重做最小调整

- 问题 2：模型层如何确定 α_t ?

本质是让“模型分布”贴近“真实数据分布”

关键洞察

看似两个问题，其实是同一件事的两个侧面

- 问题 1：数据层如何更新权重 w_{t+1} ？

本质是让权重关注“误差”的同时，对当前权重做最小调整

- 问题 2：模型层如何确定 α_t ？

本质是让“模型分布”贴近“真实数据分布”

通常用 KL 散度来衡量两个分布的差距

KL (Kullback-Leibler) 散度

$$\text{KL}(p \parallel q) = \sum_i p^i \log \frac{p^i}{q^i}$$

第一个问题：如何更新权重？

目标：在强调“没学好的样本”的前提下，对当前权重做最小调整
这可以转化为一个优化问题：

$$\min_{w_{t+1}} \text{KL}(w_{t+1} \| w_t)$$

同时加入两个约束：

- 概率约束（归一化）： $\sum_{i=1}^N w_{t+1}^i = 1$
- 偏差约束（强调“没学好的样本”）：

$$\sum_{i=1}^N w_{t+1}^i \cdot \ell(y_i, h_t(x_i)) = c$$

权重更新公式

解上述优化问题可得：

$$w_{t+1}^i \propto w_t^i \cdot \exp(\lambda_t \ell(y_i, h_t(x_i)))$$

其中拉格朗日乘子 λ_t 可数值求解

权重更新公式

解上述优化问题可得：

$$w_{t+1}^i \propto w_t^i \cdot \exp(\lambda_t \ell(y_i, h_t(x_i)))$$

其中拉格朗日乘子 λ_t 可数值求解

这正是我们想要的更新形式

- 偏差大 $\rightarrow$ 权重指数级上升
- 偏差小 $\rightarrow$ 权重变化较小

第二个问题：如何确定每棵树的权重？

现在我们有两个分布：

数据分布（真实）

$$P_{\text{data}}(x_i, y) = w_t^i \cdot 1(y = y_i)$$

模型分布

$$P_{\text{model}}(x_i, y) = w_t^i \cdot P_{\text{model}}(y | x_i)$$

其中

$$P_{\text{model}}(y | x_i) = \begin{cases} \frac{1}{1+e^{-F_t(x_i)}}, & y = 1 \\ 1 - \frac{1}{1+e^{-F_t(x_i)}}, & y = 0 \end{cases}$$

让模型分布贴近数据分布

我们最小化：

$$\min_{\alpha_t} \text{KL}(P_{\text{data}} \| P_{\text{model}})$$

展开联合分布：

$$= \sum_{i,y} w_t^i 1(y = y_i) \log \frac{w_t^i 1(y = y_i)}{w_t^i P_{\text{model}}(y|x_i)}$$

让模型分布贴近数据分布

我们最小化：

$$\min_{\alpha_t} \text{KL}(P_{\text{data}} \| P_{\text{model}})$$

展开联合分布：

$$= \sum_{i,y} w_t^i 1(y = y_i) \log \frac{w_t^i 1(y = y_i)}{w_t^i P_{\text{model}}(y|x_i)}$$

约去 w_t^i ，并消掉 $1(y = y_i)$ ：

$$= \sum_i w_t^i [-\log P_{\text{model}}(y_i|x_i)]$$

最小化 KL 散度等价于：

$$\max_{\alpha_t} \sum_i w_t^i \log P_{\text{model}}(y_i|x_i)$$

代入模型 (sigmoid):

$$P_{\text{model}}(Y = 1|x) = \frac{1}{1 + e^{-F_t(x)}}$$

得到目标函数：

$$\max_{\alpha_t} \sum_i w_t^i [y_i F_t(x_i) - \log(1 + e^{F_t(x_i)})]$$

问题已经变成一个一维优化问题

确定 α_t

代入 $F_t(x_i) = F_{t-1}(x_i) + \alpha_t h_t(x_i)$, 目标函数写为:

$$L(\alpha_t) = \sum_i w_t^i [y_i(F_{t-1}(x_i) + \alpha_t h_t(x_i)) - \log(1 + e^{F_{t-1}(x_i) + \alpha_t h_t(x_i)})]$$

这是一个关于 α_t 的单变量函数

确定 α_t

代入 $F_t(x_i) = F_{t-1}(x_i) + \alpha_t h_t(x_i)$, 目标函数写为:

$$L(\alpha_t) = \sum_i w_t^i [y_i(F_{t-1}(x_i) + \alpha_t h_t(x_i)) - \log(1 + e^{F_{t-1}(x_i) + \alpha_t h_t(x_i)})]$$

这是一个关于 α_t 的单变量函数

对 α_t 求导并令导数为 0, 得到最优条件:

$$\sum_i w_t^i h_t(x_i) [y_i - P_{\text{model}}(Y = 1|x_i)] = 0$$

这就是 α_t 的定义方程

Boosting 的本质

Boosting 本质上是在 KL 约束下

逐步构造条件概率模型的过程

加权投票只是其在对数概率空间中的线性表现

决策树的本质是信息压缩器

学习是一种压缩世界的方式